

Customer Churn Prediction System

End-to-End ML System with CRISP-DM & MLOps (MLflow)

Hector Alejandro Vargas Gutierrez

June 30, 2026

Executive Summary

- ▶ **Business Problem:** High-risk customer churn in a retail banking environment requires predictive containment.
- ▶ **Objective:** Build an automated system to flag at-risk clients and inform proactive customer retention campaigns.
- ▶ **Champion Model:** XGBoost optimized via Bayesian hyperparameter sequential optimization (Optuna TPE).
- ▶ **Validation Metric:** 5-Fold Stratified Cross-Validation Average Precision (PR-AUC) = 0.698.
- ▶ **Test Set Performance:** PR-AUC = 0.734, ROC-AUC = 0.874, Overall Accuracy = 87.3%.
- ▶ **Optimal Configuration:** RobustScaler, 1,403 trees, learning rate = 0.028, subsample = 0.97, column sample rate = 0.52, scale_pos_weight = 1.19.
- ▶ **MLOps Architecture:** Integrated end-to-end pipeline covering dynamic feature engineering, MLflow experiment state tracking, FastAPI serving layers, and automated non-parametric statistical drift monitoring.

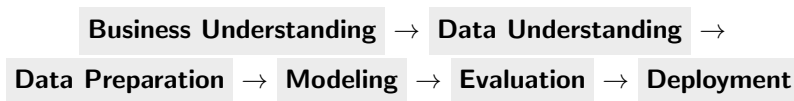
The Churn Problem in Banking

- ▶ Customer attrition directly drives revenue erosion, depresses total deposit volumes, and increases overall customer acquisition costs (CAC).
- ▶ Attrition disrupts downstream product cross-selling strategies and transaction counts.
- ▶ Early risk identification enables automated, targeted retention campaigns (incentives, customer care outreach).
- ▶ **Analytical Target:** Predict the individual conditional probability of customer departure (*Exited*).

CRISP-DM Operational Framework

Cross-Industry Standard Process for Data Mining

A structured, iterative lifecycle utilized to transform abstract business problems into verifiable production services.



Dataset Characteristics Baseline

Dataset Structural Summary

- ▶ **Total Observations:** 10,000 corporate/retail records
- ▶ **Feature Space:** 17 predictors + 1 binary target
- ▶ **Target Metric:** Exited (0 = Retained, 1 = Churned)
- ▶ **Data Quality:** Zero missing values or null inputs detected
- ▶ **Problem Type:** Imbalanced Binary Classification

Core Analytical Scope

Isolate underlying high-risk features and establish a production-ready model to predict individual attrition likelihood.

Variable Typology Mappings

Continuous Numerical Metrics

- ▶ CreditScore, Age, Tenure
- ▶ Balance, EstimatedSalary
- ▶ Point Earned, Satisfaction Score

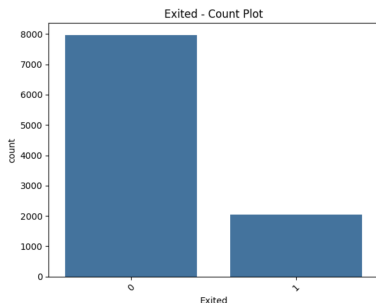
Categorical & Binary Predictors

- ▶ Geography, Gender
- ▶ Card Type (Tiers)
- ▶ HasCrCard, IsActiveMember
- ▶ Complain (Historical Flag)

Target Variable Boundary

Exited: 0 representing active retention status vs. 1 indicating confirmed customer exit.

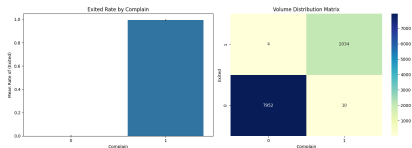
Imbalanced Target Distribution



Statistical Diagnostics

- ▶ Baseline churn rate sits at approximately **20.3%**.
- ▶ Class imbalance distorts standard accuracy metrics.
- ▶ Optimization relies strictly on Precision-Recall AUC (PR-AUC).
- ▶ Cost-sensitive adjustment implemented via XGBoost `scale_pos_weight`.

EDA Insights: Customer Complaints Impact

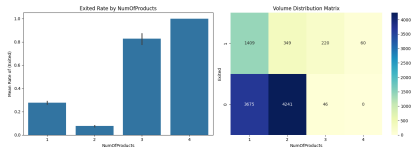


Key Risk Vector

Historical customer complaints show the strongest raw statistical association with ultimate customer exit.

- ▶ Unresolved service friction acts as an immediate attrition catalyst.
- ▶ Operationalizes customer service response time as a high-priority retention driver.

EDA Insights: Product Density Trajectories

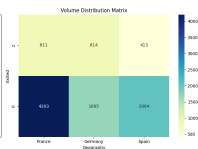
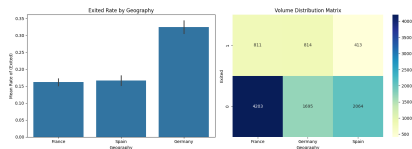


Product Congestion Anomaly

Customers holding 3 or 4 products exhibit near-total churn rates.

- Core stability rests squarely at 1 or 2 products.
- High-density product portfolios signal cross-sell friction or misaligned accounts.

EDA Insights: Macro Geographic Variations

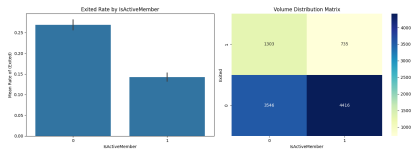


Geographic Heterogeneity

The churn rate in Germany is more than double the rates observed in France and Spain.

- ▶ Regional market variance points to localized competition or unique systemic constraints.
- ▶ Suggests a clear need for region-specific retention strategies.

EDA Insights: Platform Engagement Footprint



Engagement Variance

Inactive account status heavily correlates with verified churn.

- Drop-offs in system usage serve as an early indicator of long-term attrition.
- Proactive re-engagement campaigns should focus on reviving slipping activity metrics.

Data Preparation Architecture

- ▶ **Categorical Encoding:** One-Hot Encoding applied to multi-class features (Card Type, Geography) with `drop='first'` to safeguard against multicollinearity.
- ▶ **Continuous Scaling:** Standardization tracks continuous parameters (Balance, Age, CreditScore) to counter outliers.
- ▶ **Target Leakage Defense:** The Complain feature was removed. While it shows high correlation with churn, its inclusion causes target leakage and artificially inflates model performance.
- ▶ **Reproducibility Guardrails:** Data splits use stratified partitioning to lock down original distribution proportions across train and test states.

Sequential Feature Selection (SFS) Analytics

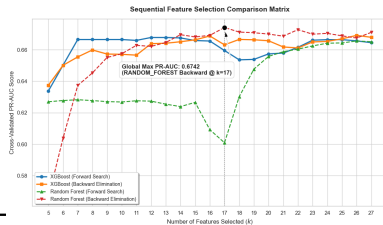
SFS Optimization Run

Sequential Forward and Backward selection loops run over baseline estimators using a 5-Fold Cross-Validation PR-AUC search framework.

SFS Exploration Outcomes

Pipeline Configuration	Features	PR-AUC
Random Forest + Backward	17	0.6742
XGBoost + Backward	16	0.6688
XGBoost + Forward	12	0.6678
Random Forest + Forward	27	0.6656

- ▶ Backward elimination consistently outperformed forward selection passes.
- ▶ Peak selection capacity stabilized at 17 features.



Optimal Feature Subset Analysis

Key Domain Findings

- ▶ Product density features are retained across all top-performing models.
- ▶ Cross-product behavioral interactions consistently outperform raw variables.
- ▶ Financial ratio features catch behavioral drop-offs much more effectively than standalone balance metrics.

SFS Champion Matrix

Backward Random Forest selection isolated **17 core features** to capture the highest baseline predictive resolution (**0.6742**).

Feature Group Categories	
Category	Extracted Feature Fields
Demographics	Gender, Germany, Spain
Product Sets	NumOfProducts_1, 2, 3, 4
Value Metrics	Salary, Point Earned, Satisfaction
Interactions	Age×IsActive, CreditScore×Age
Ratios	Balance/Product, Credit/Age
Card Tiers	Silver Card Type Flag

Baseline Model Performance Evaluation

- ▶ Diverse algorithmic architectures were evaluated using default hyperparameter baselines:
 - ▶ Linear Estimators (Logistic Regression)
 - ▶ Non-Linear Trees (Decision Tree, Random Forest)
 - ▶ Boosting Ensembles (XGBoost)
- ▶ **Primary Evaluation Metric:** Precision-Recall Area Under the Curve (**PR-AUC**).
- ▶ Proved far more robust than standard accuracy by focusing on minority positive instances over true negatives.
- ▶ **Initial Results:** Random Forest reached the highest out-of-the-box baseline score (**0.733**), followed closely by XGBoost (**0.692**).

Strategic Model Selection for Optimization

- ▶ Despite Random Forest leading with default settings, **XGBoost was chosen for the final optimization phase.**
- ▶ The baseline margin was narrow, and XGBoost offers distinct architectural advantages:
 - ▶ Fine-grained, deep hyperparameter search space customization.
 - ▶ Advanced gradient boosting convergence that typically outperforms bagging models after tuning.
 - ▶ Built-in L1 and L2 regularization parameters to protect against overfitting.
 - ▶ Highly efficient handling of complex, non-linear feature interactions.
- ▶ XGBoost presented the highest performance ceiling under Bayesian hyperparameter optimization.

Bayesian Optimization Search Framework

Optuna Hyperparameter Space Exploration Engine

Optuna's Tree-structured Parzen Estimator (TPE) sampler ran a intensive **5,000-trial search space loop** to maximize Average Precision.

- ▶ Encapsulated the entire workflow into a single robust pipeline:
 - ▶ Dynamic feature engineering extraction steps.
 - ▶ Adaptive scaler routing selections (Standard, MinMax, Robust).
 - ▶ Categorical encoding constraints and automated tree structures.
- ▶ Cross-validation used a 5-Fold Stratified strategy to enforce out-of-sample consistency.
- ▶ Optuna's MedianPruner monitored unpromising learning steps early to minimize unnecessary computational overhead.

Smart Hyperparameter Optimization with Optuna

Tree-structured Parzen Estimator (TPE)

Optuna automatically searches thousands of model configurations by learning from previous experiments and prioritizing the settings that produce better results.

- ▶ **Automated Experimentation:** Tests different combinations of model settings instead of relying on manual tuning.
- ▶ **Learning from Results:** Each experiment provides feedback, allowing Optuna to identify which configurations perform better.
- ▶ **Focused Search:** More trials are allocated to promising parameter combinations while still exploring new possibilities.
- ▶ **Efficient Model Improvement:** Finds high-performing configurations faster than testing parameters randomly.

Result: 5,000 automated trials were evaluated to identify the optimal model configuration.

TPE vs Traditional Bayesian Optimization

Different Strategies for Learning the Search Space

Both approaches use previous experiments to guide future trials, but they model the optimization problem differently.

	Classic Bayesian Optimization	Optuna TPE
Function Model	Models the objective function directly	Models parameter distributions based on trial outcomes
Common Method	Gaussian Process surrogate model	Probability density estimation
Decision Rule	Selects points with high expected improvement	Selects points where good configurations are more likely
Scalability	Can become expensive with many dimensions	Handles many hyperparameters efficiently
Best Use Case	Small search spaces with expensive evaluations	Large ML pipelines with many mixed parameters

Auditability via MLflow Experiment Tracking

Reproducible Pipeline State Governance

MLflow tracking handles full logging for every individual Optuna trial.

- ▶ Backed by a local persistent SQLite DB to guarantee run auditability.
- ▶ Structured via a clear Parent-Child run architecture:
 - ▶ **Parent Run:** Logs overall run metrics, optimized parameters, and production deployment pipeline artifacts.
 - ▶ **Child Runs:** Capture individual hyperparameter permutations from each Optuna trial.
- ▶ Automatic telemetry capture logs parameters, scores, schemas, and serialization formats.

Hyperparameter Space Search Insights

Bayesian Optimization Trajectory Convergence

The TPE search converged on the XGBoost architecture as the most effective framework for maximizing minority-class resolution.

- ▶ **Preprocessing Strategy:** Robust scaling emerged as the optimal preprocessor to manage outliers.
- ▶ **Imbalance Tuning:** Setting `scale_pos_weight = 1.19` adjusted minority-class cost parameters effectively.
- ▶ **Parameter Convergence Boundaries:**
 - ▶ Learning Rate stabilized around ≈ 0.028 .
 - ▶ Row Subsampling converged around ≈ 0.97 .
 - ▶ Column Feature Sampling centered around ≈ 0.52 .

Evaluation Metric Domain	Champion Value Result
Cross-Validation PR-AUC	0.698
Training Set PR-AUC	0.797
Independent Test Set PR-AUC	0.734
Test Generalization Accuracy	87.3%
Precision Score (Positive Predictive Value)	74.9%
Recall Score (Sensitivity)	56.4%
Balanced F1-Score	0.643
Area Under the ROC Curve (ROC-AUC)	0.874

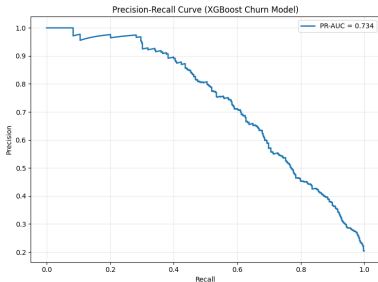
Generalization Capacity Verification

The narrow gap between train and test metrics (0.797 vs 0.734) confirms strong model generalization and rules out overfitting.

Model Diagnostics: Precision-Recall Curve

Precision-Recall Profile

- ▶ **Test PR-AUC: 0.734**
- ▶ Directly matches the evaluation metric targeted during the 5,000-trial Bayesian search.



Why PR-AUC Over ROC-AUC?

With high class imbalance, ROC-AUC can yield an overly optimistic evaluation by incorporating True Negatives. The Precision-Recall curve isolates performance on the minority class by tracking False Positives and False Negatives directly.

Operational Utility

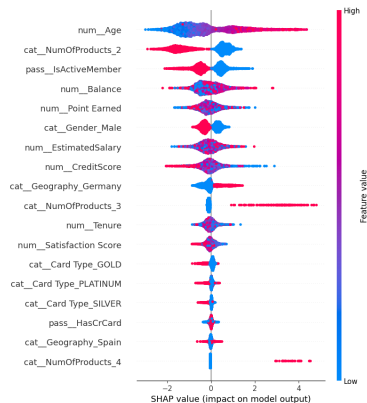
Enables business teams to dynamically adjust probability thresholds to balance marketing costs (Precision) against churn detection coverage (Recall).

Global Feature Importance Rankings

- ▶ **Feature importance** was extracted from the optimized tree-boosting matrix.
- ▶ **Product Portfolio Density** emerged as the primary driver of customer attrition:
 - ▶ NumOfProducts_2 (25.8% total attribution)
 - ▶ NumOfProducts_3 (15.4% total attribution)
 - ▶ NumOfProducts_4 (8.4% total attribution)
- ▶ **Customer Engagement Metrics** contributed significant predictive power:
 - ▶ IsActiveMember (8.7% total attribution)
 - ▶ Age (6.6% total attribution)
- ▶ **Geographic Market Location** introduced notable variation:
 - ▶ Geography_Germany (7.5% total attribution)
- ▶ The top 5 features account for roughly **66%** of the model's total predictive attribution. Card type and credit card ownership showed negligible impact.

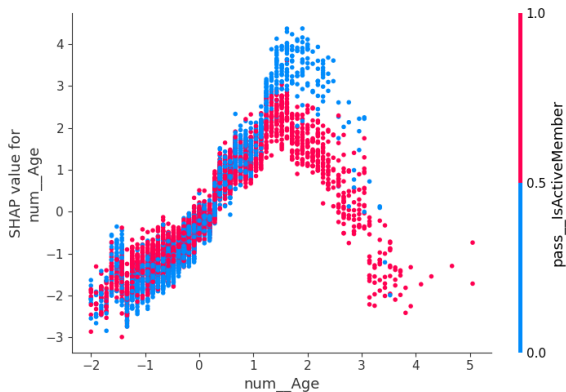
SHAP Global Interpretability Summary

- ▶ SHAP values map out individual feature impacts on directional churn risk.
- ▶ **Risk Accelerators (Increases Churn):**
 - ▶ Advancing customer age profiles.
 - ▶ Portfolios containing 3 or 4 products.
 - ▶ Residence within the German market.
 - ▶ High overall account balances.
- ▶ **Risk Decelerators (Reduces Churn):**
 - ▶ Active account membership status.
 - ▶ Portfolios holding exactly 2 products.
 - ▶ Higher baseline customer credit scores.



SHAP beeswarm summary plot for the final XGBoost model.

Age Nonlinearities: SHAP Dependence Plots



SHAP dependence mapping for Age interacting with IsActiveMember.

Age Dynamics & Interaction Insights

- ▶ Customer age displays a pronounced non-linear relationship with calculated risk metrics.
- ▶ A distinct risk transition occurs around normalized age ≈ 2.0 , where SHAP vectors shift from positive to negative.
- ▶ **Younger Segments:** Lower overall baseline attrition risk.
- ▶ **Middle-Aged Segments:** Peak risk profiles, accelerating attrition probability.
- ▶ **Older Segments:** Attrition risk drops back down, showing stable retention behavior.
- ▶ **Engagement Interaction Effect:**
 - ▶ Inactive account status amplifies the middle-age churn spike.
 - ▶ Active account status mitigates risk across all age segments.

Strategic Business Action

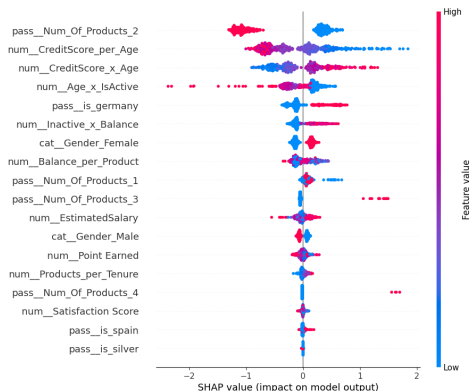
Focus retention marketing directly on middle-aged cohorts, while managing younger and older segments as low-risk baselines.

Global Impact Drivers

Global SHAP Interpretations

SHAP values isolate the absolute directional risk contribution of every feature across all observations.

- ▶ **Product Portfolio Structure:** Total product count serves as the model's primary prediction anchor.
- ▶ **Financial Profile Interactions:** Credit score combined with age provides a refined risk signal.
- ▶ **Engagement Footprints:** Inactivity paired with high account balances flags high-risk accounts.
- ▶ **Low-Impact Variables:** Estimated salary and card tier labels show minimal predictive value.



Local Interpretability: Customer Risk Case Study

High-Risk Account Breakdown

► Risk Amplifiers:

- High CreditScore $\times$ Age interaction.
- Unfavorable CreditScore-to-Age ratio.
- High balance on an inactive account.

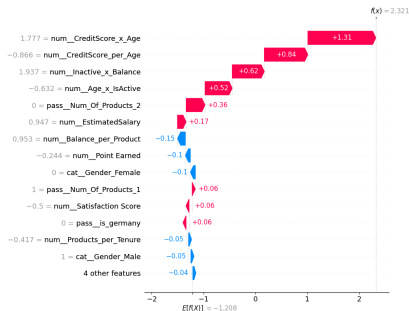
► Risk Mitigators:

- Favorable balance-to-product ratio.
- Substantial reward point accrual.

Value to Business: Provides transparent, auditable explanations to justify individual customer retention actions.

SHAP Local Waterfall Implementations

Waterfall plots break down exactly how specific features shift an individual prediction away from the baseline expected value.



Key Strategic Business Insights

- ▶ **Product Portfolio Alignment:** Product counts are the strongest indicator of churn, with multi-product combinations driving anomalies.
- ▶ **Engagement Leverage:** Active account membership remains the single most reliable hedge against customer attrition.
- ▶ **Geographic Disparities:** localized operational conditions in the German market require customized regional retention strategies.
- ▶ **Age Demographics:** Risk centers on middle-aged segments, requiring age-tailored outreach.

Priority Core Initiatives

1. Focus campaigns on re-engaging inactive accounts.
2. Optimize multi-product account groupings to prevent portfolio friction.
3. Deploy specialized retention strategies for high-risk regional markets.

Production System Architecture

Platform Components

An end-to-end MLOps ecosystem designed for dynamic feature extraction, hyperparameter shifting, and localized storage.

- ▶ **Data Ingestion Pipeline:** Handles modular ingestion via structured CSV files securely bounded by project root configurations.
- ▶ **Feature Engineering Pipeline:** `DynamicFeatureEngineer` transforms raw inputs into interaction terms and indicators.
- ▶ **Adaptive ML Optimization:** Runs sequential multi-stage Optuna tuning loops with real-time MLflow Tracking metrics.
- ▶ **Model Registry:** Tracks parameters and logs serialized cloudpickle models to a dedicated SQLite storage layer.
- ▶ **API Inference Service:** Implements microservices using FastAPI to deliver endpoints for execution and runtime statistics.

Churn Prediction Workflow

Data Lifecycle Execution

From raw transaction parameters to calculated strategic customer retention campaigns.

1. **Customer Input Data:** Raw structured vectors pass through validation checks upon reaching API micro-routers.
2. **Feature Transformation:** Preprocessors construct feature names out, scaling variables using target encodings.
3. **Pipeline Type Defense:** An implicit `XGBNumericCaster` layer forces features into `float32` shapes to shield the base boosting trees.
4. **Model Inference:** High-velocity tree tracking models run predictive passes to assign probability weights.
5. **X-Ray Explanations:** Triggers localized tree-path dependent SHAP routines to serialize feature importances back to MLflow.

Adaptive Multi-Stage Optimization Loop

- Implements sequential search boundary adjustment loops over 4 key phases to scale parameter ranges dynamically:

Optimization Phase	Trials Run	Search Boundary Strategy
Base Phase	200	Explores baseline hyperparam defaults
Phase 2 Step	100	Quantile profiling and range movement
Phase 3 Step	50	Concentrates boundaries on optimum zones
Phase 4 Step	25	Fine-tunes localized high-perf vectors

Table: Sequential Parameter Convergence Lifecycle

Model Promotion & Champion Guardrails

The tracking server queries all parent runs inside the SQLite environment. The platform automatically promotes new models only if the calculated test PR-AUC surpasses the highest historical metric.

Microservice Interface Endpoints

The delivery layer exposes clean REST schemas managed by Uvicorn.

```
# Ingestion and Retraining
@app.post("/train")
async def train(file: UploadFile, experiment_name: str):
    # Runs ingestion, engineering, and multi-stage loops

# Real-Time Operational Predictor
@app.post("/predict")
async def predict(file: UploadFile, run_id: str):
    # Transforms arrays and produces XGBoost probabilities

# Analytics and Audit
@app.get("/experiments/best-runs")
def get_best_runs():
    # Returns history tracking logs via
    # BestRunResponse Pydantic schema
```

Containerized Deployment Architecture

Reproducible Ecosystem Environment

The application is packaged inside a lightweight `python:3.12-slim` container, ensuring quick start-up times and isolation.

- ▶ **FastAPI Application Container (hop_api):** Exposes internal operational processing functions via port 8000.
- ▶ **MLflow Dashboard Service (hop_mlflow):** Mounts unified workspace directory binds to expose the visualization engine UI on port 5000.
- ▶ **Persistent Storage Layer:** Mounts localized shared system volumes to maintain tracking information inside `mlflow.db` and model plots under `mlartifacts/`.

Production Monitoring

Automated Drift Detection Framework

Production monitoring services detect data and target distribution shifts before model performance degradation becomes significant.

- ▶ **Feature Drift Detection:** KS tests for numerical variables and Chi-Square tests for categorical variables.
- ▶ **Pipeline-Aware Monitoring:** Drift evaluated after feature engineering and preprocessing transformations.
- ▶ **Target Drift Analysis:** Monitoring of churn-rate shifts and class distribution changes.
- ▶ **Monitoring APIs:** Real-time drift assessment endpoints returning statistical reports and drift alerts.

Monitoring Service Endpoints

The observability layer exposes statistical drift monitoring services for production model governance.

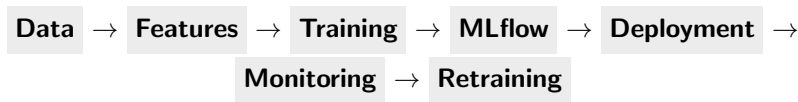
```
# Feature Distribution Monitoring
@app.post("/monitor/feature-drift")
async def monitor_drift(reference_file: UploadFile,
                        current_file: UploadFile, run_id: str):
    # Executes KS and Chi-Square drift tests
    # across transformed production features

# Target Distribution Monitoring
@app.post("/monitor/target-drift")
async def monitor_target_drift(reference_file: UploadFile,
                               current_file: UploadFile,
                               target_column: str = "Exited"):
    # Evaluates churn-rate shifts and
    # class distribution drift statistics
```

End-to-End MLOps Pipeline

Continuous Integration & Training Loop

Unified architectural lifecycle mapping feedback dependencies from collection to iteration.



Systemic & Operational Challenges

Architectural Constraints

Key operational hurdles encountered during the design, development, and deployment phases of the churn platform.

- ▶ **Class Imbalance Skew:** The minor distribution of the positive churn class ($\approx 20.3\%$) inherently biases traditional objective functions toward the dominant negative class.
- ▶ **Target Feature Leakage:** Highly predictive indicators—such as customer complaint flags—occur downstream of the initial churn signal, risking data contamination if not strictly isolated.
- ▶ **Interpretability vs. Capacity Trade-off:** High-capacity tree ensembles (XGBoost) yield superior non-linear boundaries but sacrifice clear, parametric explainability.

Implemented Engineering Mitigations

Algorithmic & Pipeline Safeguards

How the platform resolved systemic challenges to enforce production-grade reliability.

- ▶ **Cost-Sensitive Objective Weighting:** Injected an optimized `scale_pos_weight = 1.19` parameter into the booster to force the loss function to penalize minority-class misclassifications.
- ▶ **Proactive Data Isolation:** Formally stripped downstream target-leaking indicators from the registry configuration prior to modeling to guarantee an honest out-of-sample test setup.
- ▶ **Post-Hoc SHAP Layering:** Integrated localized tree-path dependent Shapley explanations to provide fully auditable, feature-level attribution without sacrificing model complexity.

Future Strategic Enhancements

Platform Evolution Roadmaps

Next-generation milestones focused on automating system scaling and enhancing operational outreach.

- ▶ **Streaming Inference Engine:** Transitioning the current microservice layer from batch payloads to event-driven, real-time scoring streams.
- ▶ **Closed-Loop Retraining Orchestration:** Automating data collection, training triggers, and champion evaluation steps via a central manager.
- ▶ **Shadow Deployment & A/B Testing Framework:** Serving alternative retention offerings to parallel customer buckets to test conversion rates under live traffic conditions.
- ▶ **Enterprise Feature Store Integration:** Centralizing feature definitions across processing environments to eliminate serving-skew anomalies.

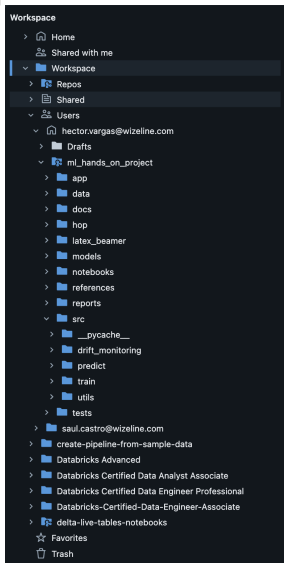
Conclusion

Platform Delivery Milestones

A comprehensive summary of the project's completed technical achievements.

- ▶ **End-to-End Delivery Accomplished:** Successfully engineered a type-safe machine learning system, moving seamlessly from raw transactional ingest to API serving structures.
- ▶ **High-Fidelity Model Performance:** Achieved a strong ****0.734 Test PR-AUC**** (Balanced F1 Score = **0.643**), outperforming baseline models and ensuring generalizability on unseen data.
- ▶ **Rigorous Model Governance:** Standardized optimization metadata recording using MLflow parent-child structures to secure reproducibility.
- ▶ **Production Integration Ready:** The containerized, microservice-driven framework is fully prepared to provide actionable decision support to executive stakeholders.

Data Bricks: Workspace



Data Bricks: Github connection

ml_hands_on_project  [Send feedback](#)

 Branch: databricks 

Create Branch [66ebc85](#) 

Changes

Settings

Repo ID
1359497575636283 

Git repository URL 

Git provider 

Git credential 



 (hector.vargas@wizelin...)

☐ Sparse checkout mode 

Data Bricks: Catalog

The screenshot displays the Data Bricks Catalog interface. On the left, a sidebar shows a tree view of the catalog structure, including 'data Serverless', 'Catalog', and various data sources like 'bronze_wanderbricks' and 'havq_data'. The 'volumen' volume is selected under 'havq_data'. The main panel shows the 'volumen' volume details, including a description, a list of files, and a table of file metadata.

Catalog Explorer > datacatalog_dbx > havq_data > volumen

Overview Files Details Permissions

Description

This volume is used for storing data assets related to the 'havq_data' schema within the data platform. It typically contains structured data files such as Parquet, CSV, or Delta Lake formats that support analytical workloads. Teams access this volume to read and write batch and streaming data as part of ETL pipelines, data analysis, and reporting tasks. The data stored here usually serves as source or intermediate datasets for downstream processing and querying.

About this volume

Owner: Hector Vargas Gutierrez

Tags: Add tags

Files

Name	Size	Last modified
X_test.csv	128.82 KB	10 hours ago
X_train.csv	516.94 KB	10 hours ago
X_val.csv	159.36 KB	13 hours ago
predictions.csv	34.96 KB	3 hours ago
selected_features.json	479.00 B	9 hours ago
train_test_dataset.csv	652.37 KB	7 days ago
y_test.csv	3.13 KB	10 hours ago
y_train.csv	12.51 KB	10 hours ago
y_val.csv	3.91 KB	13 hours ago

Data Bricks: Jobs Pipelines

The screenshot displays the 'Jobs & Pipelines' section of the Data Bricks platform. At the top, there are tabs for 'Jobs & pipelines' and 'Runs'. Below this, a 'Create new' section offers three options: 'Ingestion pipeline' (Ingest data from apps, databases and files), 'ETL pipeline' (Build ETL pipelines using SQL and Python), and 'Job' (Orchestrate notebooks, pipelines, queries and more). A search bar and filter tabs (All, Jobs, Pipelines, Owned by me, Accessible by me, Favorites, Tags, Run as) are present. The main table lists the following items:

Name	Type	Tags	Run as	Trigger	Recent runs
dbdemo5_dt_lakehouse_churn_datacartel_dbx_dbd...	ETL pipeline		Victor Regueira Romero		— — — — —
★ hop_predict_pipeline_job	Job		Hector Vargas Gutierrez		🔴 🔴 🔴 🔴 🔴
★ hop_training_pipeline_job	Job		Hector Vargas Gutierrez		🔴 🔴 🔴 🔴 🔴

Each row in the table includes action icons (run, edit, delete) and a 'Recent runs' column with status indicators (red for failed, green for successful). The interface also includes a 'Send feedback' link and a 'Create' button.

Data Bricks: Job Training Runs

The screenshot shows the Data Bricks Jobs & Pipelines interface for a job named `hop_training_pipeline_job`. The interface is divided into several sections:

- Jobs & Pipelines:** At the top, showing the job name and a "Send feedback" link.
- Runs:** A section on the left showing a bar chart of run durations. The chart has two bars: one for "Ingest" (1h 15m 31s) and one for "feature_engineering" (37m 45s). Below the chart, a table lists the tasks: "Ingest", "feature_engineering", and "model_optimization".
- Task Details:** A section on the right showing details for the selected task. It includes the Job ID (873888329449882), the Creator (Hector Vargas Gutierrez), and the Run as (Hector Vargas Gutierrez). It also shows the Description (Add description) and Lineage (No lineage information for this job).
- Schedules & Triggers:** A section on the right showing the schedule (None) and the trigger (Add trigger).
- Job parameters:** A section on the right showing the job parameters (No job parameters are defined for this job).
- Compute:** A section on the right showing the compute configuration (Job_cluster, Driver: rd-fleet.xlarge, Workers: rd-fleet.xlarge - 8 workers - On-demand and Spot - fail back to On-demand - DBR: 17.3 LTS (includes Apache Spark 4.0.0, Scala 2.13) - us-west-2c).

Below the task details, there is a table of runs. The table has columns: Start time, Run ID, Launched, Duration, Status, Error code, and Run parameters. The table shows a list of runs, with the most recent run (Run ID: 4324195689052058) being the selected one.

Start time	Run ID	Launched	Duration	Status	Error code	Run parameters
Jun 30, 2026, 01:09 AM	638672330246078	Manually	1h 15m 32s	⚡ Succeeded		
Jun 29, 2026, 06:19 PM	963012466246013	Manually	54m 20s	⚡ Succeeded		
Jun 29, 2026, 04:54 PM	146962044400069	Manually	1h 1m 20s	⚡ Succeeded		
Jun 29, 2026, 03:43 PM	469571126313476	Manually	48m 1s	⚡ Succeeded		
Jun 29, 2026, 03:19 PM	856357027034966	Manually	10m 29s	❌ Failed	RunExecutionError	
Jun 29, 2026, 02:51 PM	4324195689052058	Manually	6m 9s	⚡ Succeeded		

Data Bricks: Job Training Tasks

Jobs & Pipelines >

 **hop_training_pipeline_job** ★ [Send feedback](#)

Runs **Tasks**

ingest
Python file at ...ct/src/train/ingest.py
Data Team Shared Cluster

feature_engineering
Python file at ...ature_engineering.py
Data Team Shared Cluster

model_optimization
Python file at ...odel_optimization.py
Data Team Shared Cluster

[+ Add task](#)

Task name*  model_optimization

Type* Python script 

Source*  Workspace 

Path*  ...rgas@wizeline.com/ml_hands_on_project/src/train/model_optimization.py  

Cluster*  Data Team Shared Cluster DBR 16.4 LTS · Spark 3.5.2 · Scala 2.13  

 Jobs running on all-purpose clusters are considered all-purpose compute. [Learn more](#)

Depends on feature_engineering  

Data Bricks: Job Predict Tasks

The screenshot displays the Data Bricks Jobs & Pipelines interface. At the top, the breadcrumb 'Jobs & Pipelines' is followed by the job name 'hop_predict_pipeline_job' with a star icon and a 'Send feedback' link. Below this, the 'Runs' tab is selected, and the 'Tasks' sub-tab is active. The task configuration area shows two tasks: 'run_predictions' and 'register_model'. 'run_predictions' is a Python script located at '.../predict_pipeline.py' on the 'Data Team Shared Cluster'. It is connected to 'register_model' by an arrow. Below the task list, a '+ Add task' button is visible. The configuration form for 'run_predictions' includes the following fields:

- Task name***: run_predictions
- Type***: Python script
- Source***: Workspace
- Path***: .../args@wizeline.com/ml_hands_on_project/src/predict/predict_pipeline.py
- Cluster***: Data Team Shared Cluster (DBR 16.4 LTS · Spark 3.5.2 · Scala 2.13). A note indicates: 'Jobs running on all-purpose clusters are considered all-purpose compute. [Learn more](#)'.
- Depends on**: Select task dependencies...
- Dependent libraries**: ...kspase/Users/hector.vargas@wizeline.com/ml_hands_on_project/requirements.txt
- Parameters**: {"models":["/m-ce0850f7c6de4b38870286507e393389"]}
- Notifications**: + Add
- Retries**: + Add
- Metric thresholds**: + Add

Navigation icons are visible at the bottom right of the interface.

Data Bricks: SQL Queries

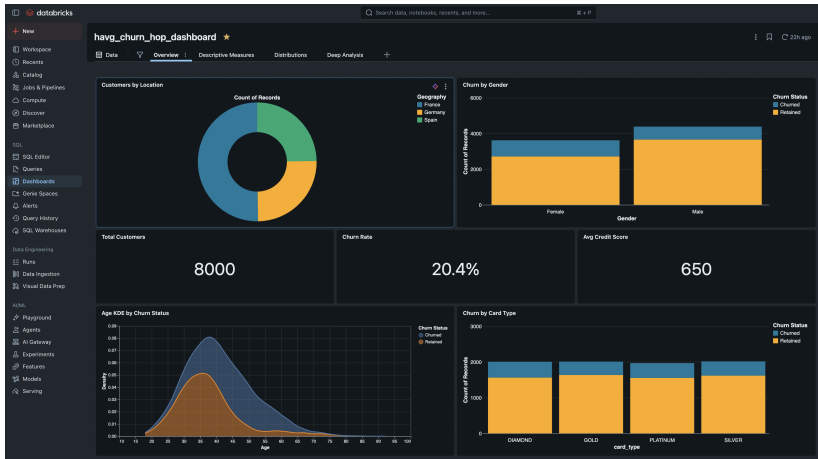
The screenshot displays the Databricks SQL Editor interface. On the left is a sidebar with navigation options: New, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor (selected), Queries, Dashboards, Genie Spaces, Alerts, Query History, and SQL Warehouses. The main area is divided into three panes. The top-left pane shows a tree view of the workspace with folders like Drafts and ml_hands_on_project, and files like databricks, churn_query_select_all, databricks_experiments_v1, databricks_experiments_v2, and havg_churn_hop_dashboard. The top-right pane shows the query editor with a 'Run all (1000)' button and a status bar indicating '23 hours ago (2h)' and 'datacartel_dbx. @_data_classification'. The bottom pane shows the results of the query, which is a table with one row and one column.

```
1 CREATE OR REPLACE TABLE train_test_dataset AS
2 SELECT
3   'Satisfaction Score' AS satisfaction_score,
4   'Card Type' AS card_type,
5   'Point Earned' AS point_earned,
6   *
7 EXCEPT ('Satisfaction Score', 'Card Type', 'Point Earned')
8 FROM read_files(
9   '/Volumes/datacartel_dbx/havg_data/volume/train_test_dataset.csv',
10  format => 'csv',
11  header => true
12 );
13
14 SELECT COUNT(*) FROM train_test_dataset;
```

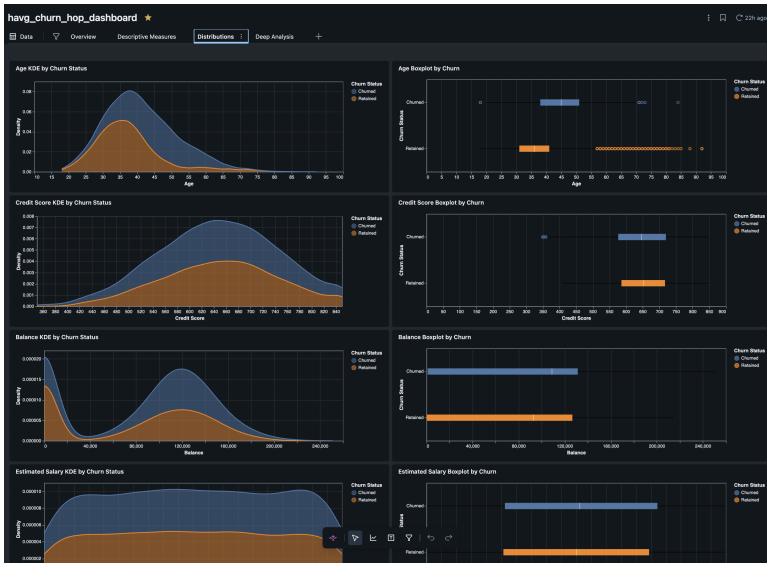
Results 2 of 2

count(1)
8000

Data Bricks: Dashboard 1



Data Bricks: Dashboard 2



Data Bricks: Experiments

Experiments

 GenAI apps & agents
Build a Generative AI application or agent

 Forecasting
Build a forecasting model using AutoML

 Classification
Build a classification model using AutoML

 Regression
Build a regression model using AutoML

 Custom model training
Train a custom classical or

MLflow 3 is available!

Featuring unified ML and GenAI experiment tracking, improved model logging, prompt versioning, enhanced LLM judges, advanced tracing for end-to-end agent observability, and beyond. [Learn more about ML features](#) | [Learn more about GenAI](#)

Filter experiments

Q

Only my experiments

Name	Created by	Last modified	Location
databricks_experiments_v2	 Hector Vargas Gutierrez	2026-06-30 02:24:18 CST	/Users/hector.vargas@wizeline.com/databricks_exper
databricks_experiments_v1	 Hector Vargas Gutierrez	2026-06-30 00:50:51 CST	/Users/hector.vargas@wizeline.com/databricks_exper
customer_churn_mock	 Victor Regueira Romero	2025-04-24 10:35:45 CST	/Shared/0bdemos/experiments/lakehouse-retail-c360

Data Bricks: Register Model 1

Registered Models				
Share and serve machine learning models. Learn more G				
Filter models Owned by me Owner ▾				
Name	Catalog	Schema	Last modified	Owner
customerchurngbclassifier	datacatalog_dbx	hang_data	Jun 30, 2026, 07:42 AM CST	 Hector Vargas Gutierrez
tpe_base_en_v1_5	system	ai	Jun 29, 2026, 12:26 PM CST	
dbdemo_customer_churn	datacatalog_dbx	dbdemo_retail_c360	Apr 24, 2025, 10:35 AM CST	 Victor Reguerra Romero
toxicity_predictor	datacatalog_dbx	default	May 11, 2025, 08:58 PM CST	 Cristian Galindo Gonzalez
gemma-3-12b-it	system	ai	Jun 29, 2026, 12:26 PM CST	
llama_v3_2_3b	system	ai	Jun 29, 2026, 12:26 PM CST	

Data Bricks: Register Model 2

Catalog Explorer > datacatalog_dbx > havg_data >

customerchurnxgbclassifier  

Overview Details Permissions

Description 

This model is designed to predict customer churn, which refers to the likelihood of customers ceasing to do business with a company. The primary output is a binary classification indicating whether a customer is likely to churn or not, based on input features derived from customer behavior data, transaction history, and demographic information. Expected inputs include customer-specific data points such as age, tenure, account type, usage patterns, and other relevant metrics. The model processes these variables to provide a score that indicates the risk of churn for each customer.

This model is appropriate to use when businesses want to proactively identify at-risk customers and implement strategies to retain them, such as targeted marketing efforts or personalized outreach. It can be integrated into workflows by automating churn predictions at regular intervals or in response to specific events, allowing teams to allocate resources effectively to address customer needs.

[Show less](#)

Versions

Status	Version	Tags	Aliases	Deployment job status	Active endpoints	Comment
	Version 3				havg_churn_model_v2	
	Version 2					
	Version 1				havg_churn_model	

Activity log

Activity logs capture updates from deployment jobs. [Learn more](#)

Time	Version	Log
2028-06-30 07:42:57	Version 3	Model version 3 registered by user hector.vargas@wizeline.com.
2028-06-30 02:24:42	Version 2	Model version 2 registered by user hector.vargas@wizeline.com.
2028-06-29 23:21:28	Version 1	Model version 1 registered by user hector.vargas@wizeline.com.

About this model

Owner [Hector Vargas Gutierrez](#)

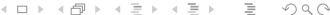
Deployment job

To manage the model lifecycle, link a deployment job. A deployment job can automatically trigger steps such as evaluation, approval, and deployment each time a new model version is created. [Learn more](#)

[Connect deployment job](#)

Tags

[Add tags](#)



Data Bricks: Register Model 3

```
%pip install "mlflow-skinny[databricks]>=2.4.1"  
dbutils.library.restartPython()
```

```
import mlflow  
catalog = "datacartel_dbx"  
schema = "havg_data"  
model_name = "customerchurnxgbclassifier"  
model_uri = "models:/m-ced050f7c6de4b38870286507e393389"  
mlflow.set_registry_uri("databricks-uc")  
mlflow.register_model(model_uri=model_uri, name=f"{catalog}.{schema}.{model_name}")
```

Data Bricks: Model Serving 1

Serving endpoints [Send feedback](#) [Create serving endpoint](#)

[Integrate coding ...](#) [GPT-5.5](#) [Claude Opus 4.7](#) [Claude Opus 4.8](#) [Gemini 3.5 Flash](#) [GPT-5.4](#) [GPT-5.5 Pro](#) [GPT-5.4 Mini](#) [GPT-5.4 Nano](#) [6](#)

[Get Started](#) [Use](#) [Copy](#) [Use](#) [Copy](#) [Use](#) [Copy](#) [Use](#) [Copy](#) [Use](#) [Copy](#) [Use](#) [Copy](#) [Use](#) [Copy](#) [Use](#) [Copy](#) [Use](#)

[Owned by me](#)

Name	State	Served entities	Tags	Task	Created by	Last modified	
hang_churn_model_v2	Ready	deparcnet_0to_hang_data_customerchurnmodel_v2 [v 3]			Hector Vargas Gutierrez	3 hours ago	
hang_churn_model	Not ready (Update failed)	No served entities			Hector Vargas Gutierrez	12 hours ago	
databricks-claude-opus-4-8	Ready	Claude Opus 4.8		Chat		1 month ago	
databricks-gemini-3-5-flash	Ready	Gemini 3.5 Flash		Chat		1 month ago	

Data Bricks: Model Serving 2

Serving endpoints >

havg_hop_churn_model_v2

Overview Metrics Deployments Logs

Ready https://dbc-6ed3f6aa-5d5b.cloud.databricks.com/serving-endpoints/havg_hop_churn_model_v2/invocations

AI Gateway Edit AI Gateway

Usage monitoring: system.serving.endpoint_usage Rate limits: Not enabled

Dimension table: system.serving.served_entities

Inference tables and telemetry: Preview Not enabled

Active configuration

Entity	Version	Name	State	Compute	Traffic (%)
datacartel_dbx_havg_data.customerchurngbclassifier	Version 3	customerchurngbclassifier-3	Ready (Scaled to zero)	CPU (4 GB/worker), Custom 0-4 concurrency	100

View starter code

cURL Python SQL

```
curl \
-u token:$DATABRICKS_TOKEN \
-X POST \
-H "Content-Type: application/json" \
-d@data.json \
https://dbc-6ed3f6aa-5d5b.cloud.databricks.com/serving-endpoints/havg_hop_churn_model_v2/invocations
```

Try in Browser

Data Bricks: Model Serving 3

```
▶ ✓ 4 minutes ago (<1s) 1 Python ✕ ⌕ ⌕ ⌕ ⌕  
  
import requests  
url = "https://dbc-6ed3f6aa-5d5b.cloud.databricks.com/serving-endpoints/hava_hop_churn_model_v2/invocations"  
  
headers = {  
    "Authorization": f"Bearer {dbutils.notebook.entry_point  
    .getDbutils().notebook().getContext().apiToken().get()}",  
    "Content-Type": "application/json"  
}
```

columns

▶ satisfaction_score
▶ card_type
▶ point_earned
▶ RowNumber
▶ CustomerId
▶ Surname
▶ CreditScore
▶ Geography
▶ Gender

values

▶ 2
▶ "Silver"
▶ 300
▶ 123456
▶ 15634602
▶ "Hargrave"
▶ 200
▶ "Spain"
▶ "Male"

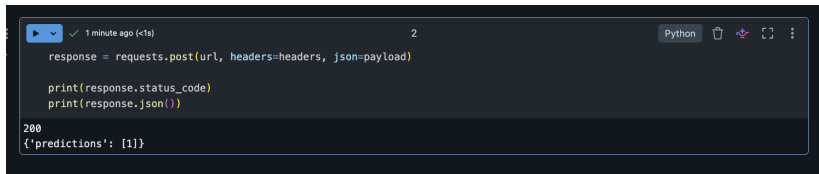
columns

▶ Age
▶ Tenure
▶ Balance
▶ NumOfProducts
▶ HasCrCard
▶ IsActiveMember
▶ EstimatedSalary
▶ Complain
▶ _rescued_data

values

▶ 35
▶ 2
▶ 5500.0
▶ 4
▶ 0
▶ 0
▶ 1010.88
▶ 0
▶ None

Data Bricks: Model Serving 4



The screenshot shows a single code cell in a Data Bricks notebook. The cell has a blue play button icon, a green checkmark, and a timestamp of "1 minute ago (<1s)". The code is written in Python and uses the 'requests' library to perform a POST request. The output of the cell is a status code of 200 and a JSON response containing a list with one element.

```
response = requests.post(url, headers=headers, json=payload)

print(response.status_code)
print(response.json())
```

200
{'predictions': [1]}

Thank You

Questions & Answers Session

`hector.vargas@wizeline.com`